

LC 62: Unique Paths

(BNY) | (DE Shaw) | (Jio)

© Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 28, 2026

Problem Understanding

- Need to go from **[0][0]** to **[m-1][n-1]**.
- And return the number of possible unique paths.
- We need to think, how many ways can reach each cell.
- If Aadya is at **(i, j)**, she could have only come from above **(i-1, j)** or from left **(i, j-1)**.
- Because it's given she can only go **down & right**.

```
ways(i, j) = ways(top) + ways(left)
```

Approach 1 → DFS

At every cell, 2 choices: **go right / go down**.

So $dfs(r,c) \rightarrow O(2^{m+n})$

Slow because some states are recomputed.

Bruteforce DFS Code:

```
class Solution {
public:
    int m;
    int n;
    int dfs(int r,int c){
        if(r>n || c>=m) return 0;
        if(r==n-1 && c==m-1) return 1;

        int down=dfs(r+1,c);
        int right=dfs(r,c+1);
        return down+right;
    }
    int uniquePaths(int mm, int nn) {
        m=mm;
        n=nn;
        return dfs(0,0);
    }
};
```

Approach 2 → DFS + Memoization

TC = $O(m \times n)$

Store $dp[r][c]$ → if already computed return directly.

Algorithm for DFS + Memoization:

- | S1: start dfs(0,0)
- | S2: base cases → invalid row, already visited, reach destination, otherwise
- | S3: $dp[r][c] = \text{left} + \text{up}$
- | S4: return $dp[r][c]$

DFS + Memoization Code:

```
class Solution {
public:
    int m;
    int n;
    vector<vector<int>> dp;
    int dfs(int r, int c) {
        if (r >= n || c >= m) return 0;
        if (r == n-1 && c == m-1) return 1;
        if (dp[r][c] != -1) return dp[r][c];

        int down = dfs(r+1, c);
        int right = dfs(r, c+1);
        dp[r][c] = down + right;
        return dp[r][c];
    }
    int uniquePaths(int nn, int mm) {
        n = nn;
        m = mm;
        dp.assign(n, vector<int>(m, -1));
        return dfs(0, 0);
    }
};
```

Approach 3 → Bottom up DP

TC = $O(m \times n)$

For $dp[i][j]$ use **top + left** → $dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Visualizing the DP Grid (Base Cases)

Base case → First row (cause you can't go up) and First column (cause you can't go left).

1 (Base)	1 (Base)	1 (Base)	1 (Base)
1 (Base)	dp[1][1]	dp[1][2]	dp[1][3]
1 (Base)	dp[2][1]	dp[2][2]	dp[2][3]

Algorithm:

S1: make DP grid

S2: fill first row, first col with 1

S3: for all remaining cells:

```
int up = dp[i-1][j]
int left = dp[i][j-1]
dp[i][j] = up + left
```

S4: return dp[m-1][n-1]

DP Approach Code:

```
class Solution {
public:
    int uniquePaths(int n, int m) {
        vector<vector<int>> dp(n, vector<int>(m, 0));
        for(int i=0; i<n; i++){
            dp[i][0]=1;
        }

        for(int j=0; j<m; j++){
            dp[0][j]=1;
        }

        for(int i=1; i<n; i++){
            for(int j=1; j<m; j++){
                dp[i][j]=dp[i-1][j]+dp[i][j-1];
            }
        }

        return dp[n-1][m-1];
    }
};
```